

Reto 3 Enunciado:

Implementa una base de datos en ORACLE sobre un tema que te interese. Debe cumplir:

- Es necesario que implemente relaciones 1:1, 1:N y M:N.
- Se deben usar todos los tipos de datos vistos en clase.
- Se insertarán al menos 3 registros en cada tabla mediante SQL.

1.- Realizar el sistema de tablas y la inserción de datos. (2 PUNTOS)

- Se implementa una base de datos que refleja un negocio de venta de packs fotográficos compuestos por cámara y objetivo; la base de datos recogerá las siguientes tablas con las relaciones descritas a continuación:

> Tabla **PACKS_FOTOGRAFIA**, que recoge los tipos de pack a la venta, con los campos: etiquetado, marca, cámara, objetivo, precio y fecha_creacion.

> Tabla **STOCKS**, que recoge el stock de cada tipo de pack puesto a la venta, con los campos: id_pack (identificando el pack correspondiente) y stock con el número de unidades de cada uno en almacén.

> Tabla **CLIENTES**, que recoge las empresas que realizan pedidos de packs fotográficos, con los campos: id_empresa y nombre.

> Tabla **PEDIDOS**, que recoge los campos: id_pedido, id_empresa, id_pack e importe; cada pedido puede contener varios packs, por lo que cada pack que conforme el mismo dará lugar a un registro en la tabla.

→ Para todas las tablas se ha implementado un campo ID autoincremental.

- Las relaciones entre las entidades correspondientes a las tablas son las siguientes:

- | | |
|--|-------------------------|
| > PACKS_FOTOGRAFIA y STOCKS | → relación 1:1 . |
| > CLIENTES y PEDIDOS | → relación 1:N . |
| > PACKS_FOTOGRAFIA y PEDIDOS | → relación M:N . |

- PEGA AQUÍ LAS SENTENCIAS DDL Y DML -

> Creación de la tabla **PACKS_FOTOGRAFIA** e inserción de registros correspondientes:

```
CREATE TABLE PACKS_FOTOGRAFIA(  
  ID NUMBER(4) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  ETIQUETADO CHAR(6) NOT NULL UNIQUE,  
  MARCA VARCHAR2 (20) NOT NULL,  
  CAMARA VARCHAR2 (20) NOT NULL,  
  OBJETIVO VARCHAR2 (50) NOT NULL,  
  PRECIO NUMBER(6,2),  
  FECHA_CREACION DATE  
);  
  
INSERT INTO PACKS_FOTOGRAFIA (ETIQUETADO,MARCA,CAMARA,OBJETIVO,PRECIO,FECHA_CREACION)  
VALUES ('PA0000','Valor Control',' Valor Control ',' Valor Control ',0,TO_DATE('2000-01-01','YYYY-MM-DD'));  
INSERT INTO PACKS_FOTOGRAFIA (ETIQUETADO,MARCA,CAMARA,OBJETIVO,PRECIO,FECHA_CREACION)  
VALUES ('PA0001','NIKON','D7000','NIKKOR A DX 12-28MM PZ VR',655.95,TO_DATE('2021-05-13','YYYY-MM-DD'));  
INSERT INTO PACKS_FOTOGRAFIA (ETIQUETADO,MARCA,CAMARA,OBJETIVO,PRECIO,FECHA_CREACION)  
VALUES ('PA0002','NIKON','D7200','NIKKOR AF-S DX VR 18-105MM G',430.99,TO_DATE('2022-03-11','YYYY-MM-DD'));  
INSERT INTO PACKS_FOTOGRAFIA (ETIQUETADO,MARCA,CAMARA,OBJETIVO,PRECIO,FECHA_CREACION)  
VALUES ('PA0003','CANON','EOS 250D','EF 100-400 mm - f/4.5-5.6 L IS USM',1050.95,TO_DATE('2020-01-10','YYYY-MM-DD'));
```

> Creación de la tabla **STOCKS** e inserción de registros correspondientes:

```
CREATE TABLE STOCKS(  
  ID NUMBER(4) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  ID_PACK NUMBER(4) NOT NULL UNIQUE,  
  STOCK NUMBER(4),  
  FOREIGN KEY (ID_PACK) REFERENCES PACKS_FOTOGRAFIA (ID)  
);  
  
INSERT INTO STOCKS (ID_PACK,STOCK) VALUES (1,15);  
INSERT INTO STOCKS (ID_PACK,STOCK) VALUES (2,1524);  
INSERT INTO STOCKS (ID_PACK,STOCK) VALUES (3,259);
```

> Creación de la tabla **CLIENTES** e inserción de registros correspondientes:

```
CREATE TABLE CLIENTES(  
  ID NUMBER(4) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  ID_EMPRESA VARCHAR2(5) NOT NULL UNIQUE,  
  NOMBRE VARCHAR2 (50)  
);  
  
INSERT INTO CLIENTES (ID_EMPRESA,NOMBRE) VALUES ('E0000','Ejemplo control');  
INSERT INTO CLIENTES (ID_EMPRESA,NOMBRE) VALUES ('E0001','TUS FOTOS S.L.');
```

```
INSERT INTO CLIENTES (ID_EMPRESA,NOMBRE) VALUES ('E0002','CAMARÓGRAFOS SIN FRONTERAS');  
INSERT INTO CLIENTES (ID_EMPRESA,NOMBRE) VALUES ('E0003','FOTOBODAS');
```

> Creación de la tabla **PEDIDOS** e inserción de registros correspondientes:

```
CREATE TABLE PEDIDOS (  
  ID NUMBER(6) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  ID_PEDIDO CHAR (20) NOT NULL,  
  ID_EMPRESA NUMBER(4) NOT NULL,  
  ID_PACK NUMBER(4) NOT NULL,  
  IMPORTE NUMBER(7,2) NOT NULL,  
  FOREIGN KEY (ID_EMPRESA) REFERENCES CLIENTES(ID),  
  FOREIGN KEY (ID_PACK) REFERENCES PACKS_FOTOGRAFIA(ID)  
);  
  
INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000001', 2, 1, 1086.99);  
INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000001', 2, 2, 1086.99);  
INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000002', 1, 1, 2137.89);  
INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000002', 1, 2, 2137.89);  
INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000002', 1, 3, 2137.89);  
INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000003', 1, 3, 1050.95);
```

2.- Generar las vistas que unan todas las tablas. (3 PUNTOS)

PEGAR EL CÓDIGO SQL DE CREACIÓN DE CADA VISTA. SE PIDE UNA VISTA CON LA RELACIÓN 1:N, OTRA CON LA RELACIÓN M:N Y OTRA CON TODO EL SISTEMA COMPLETO. USA INNER JOIN.

- PEGA AQUÍ LAS SENTENCIAS SQL -

> Creación de la vista de la unión de las tablas para la relación **1:N** → **CLIENTES** y **PEDIDOS**

```
CREATE VIEW CLIENTES_PEDIDOS AS  
SELECT C.NOMBRE AS EMPRESA, P.ID_PEDIDO AS "ID PEDIDO", P.IMPORTE AS "IMPORTE PEDIDO(€)"  
FROM CLIENTES C INNER JOIN PEDIDOS P  
ON C.ID = P.ID_EMPRESA;
```

> Creación de la vista de la unión de las tablas para la relación **M:N** → **PACKS_FOTOGRAFIA** y **PEDIDOS**

```
CREATE VIEW PEDIDOS_PACKS AS  
SELECT P.ID_PEDIDO AS "ID PEDIDO", P.ID_EMPRESA AS "ID EMPRESA", PF.ETIQUETADO AS PACK, PF.CAMARA, PF.OBJETIVO,  
P.IMPORTE AS "TOTAL PEDIDO (€)"  
FROM PEDIDOS P INNER JOIN PACKS_FOTOGRAFIA PF  
ON P.ID_PACK = PF.ID;
```

> Creación de la vista de la unión de todas las tablas

```
CREATE VIEW TODAS_TABLAS AS  
SELECT P.ID_PEDIDO AS "ID PEDIDO", C.NOMBRE AS CLIENTE, PF.ETIQUETADO AS PACK, PF.CAMARA, PF.OBJETIVO,  
P.IMPORTE AS "TOTAL PEDIDO (€)", S.STOCK  
FROM PEDIDOS P INNER JOIN PACKS_FOTOGRAFIA PF  
ON P.ID_PACK = PF.ID  
INNER JOIN STOCKS S  
ON PF.ID = S.ID_PACK  
INNER JOIN CLIENTES C  
ON P.ID_EMPRESA = C.ID  
ORDER BY P.ID_PEDIDO;
```

- PEGA AQUÍ LAS CAPTURA DE PANTALLA COMPLETAS QUE MUESTREN SU CORRECTA EJECUCIÓN -

> Vista de la unión de las tablas para la relación 1:N → CLIENTES y PEDIDOS

The image displays two screenshots of the Oracle FreeSQL web interface, demonstrating the successful execution of SQL scripts to create a view and query its results.

Top Screenshot: Shows the SQL script execution. The script in the editor is:

```
1 CREATE VIEW CLIENTES_PEDIDOS AS
2 SELECT C.NOMBRE AS EMPRESA, P.ID_PEDIDO AS "ID PEDIDO", P.IMPORTE AS "IMPORTE PEDIDO(€)"
3 FROM CLIENTES C INNER JOIN PEDIDOS P
4 ON C.ID = P.ID_EMPRESA;
5
```

The "Query result" tab shows the following output:

```
SQL> CREATE VIEW CLIENTES_PEDIDOS AS
SELECT C.NOMBRE AS EMPRESA, P.ID_PEDIDO AS "ID PEDIDO", P.IMPORTE AS "IMPORTE PEDIDO(€)"
FROM CLIENTES C INNER JOIN PEDIDOS P
ON C.ID = P.ID_EMPRESA

View CLIENTES_PEDIDOS created.
Elapsed: 00:00:00.012
```

Bottom Screenshot: Shows the query results for the view. The script in the editor is:

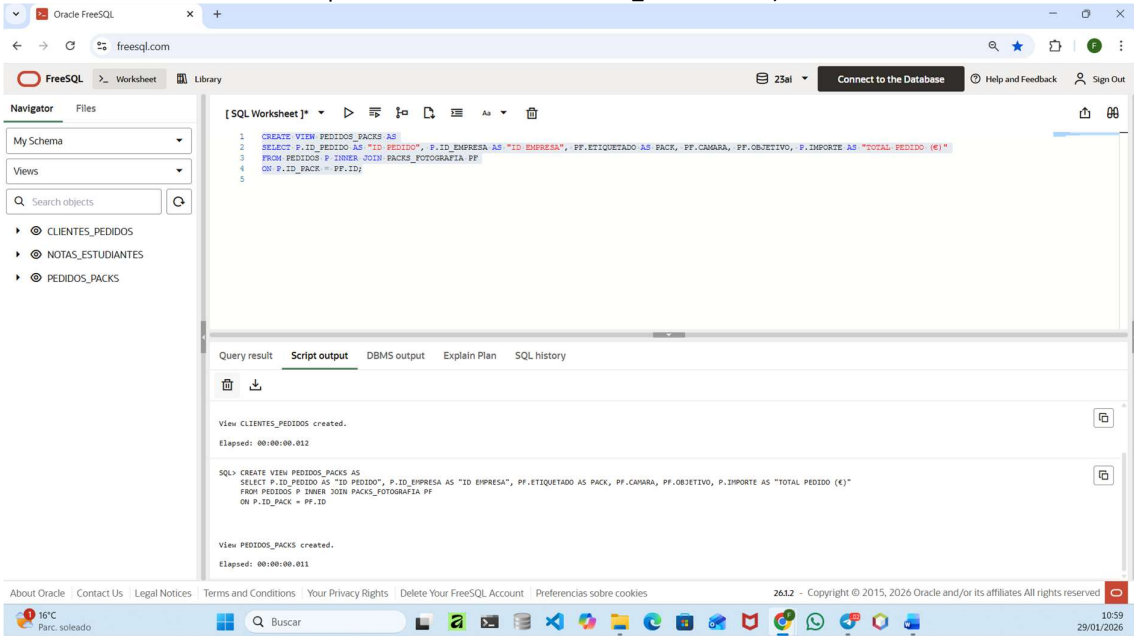
```
1 CREATE VIEW CLIENTES_PEDIDOS AS
2 SELECT C.NOMBRE AS EMPRESA, P.ID_PEDIDO AS "ID PEDIDO", P.IMPORTE AS "IMPORTE PEDIDO(€)"
3 FROM CLIENTES C INNER JOIN PEDIDOS P
4 ON C.ID = P.ID_EMPRESA;
5
6 SELECT * FROM CLIENTES_PEDIDOS;
```

The "Query result" tab shows the following output:

```
Download Execution time: 0.081 seconds
```

	EMPRESA	ID PEDIDO	IMPORTE PEDIDO(€)
1	TUS FOTOS S.L.	PE00000000000000000001	1086.99
2	TUS FOTOS S.L.	PE00000000000000000001	1086.99
3	Ejemplo control	PE00000000000000000002	2137.89
4	Ejemplo control	PE00000000000000000002	2137.89
5	Ejemplo control	PE00000000000000000002	2137.89
6	Ejemplo control	PE00000000000000000003	1050.95

> Vista de la unión de las tablas para la relación M:N → PACKS_FOTOGRAFIA y PEDIDOS



Oracle FreeSQL

Navigator

My Schema

Views

Search objects

CLIENTES_PEDIDOS

NOTAS_ESTUDIANTES

PEDIDOS_PACKS

[SQL Worksheet]

```
1 CREATE VIEW PEDIDOS_PACKS AS
2 SELECT P.ID_PEDIDO AS "ID PEDIDO", P.ID_EMPRESA AS "ID EMPRESA", PF.ETIQUETADO AS PACK, PF.CAMARA, PF.OBJETIVO, P.IMPORTE AS "TOTAL PEDIDO (€)"
3 FROM PEDIDOS P INNER JOIN PACKS_FOTOGRAFIA PF
4 ON P.ID_PACK = PF.ID;
5
```

Query result

Script output

DBMS output

Explain Plan

SQL history

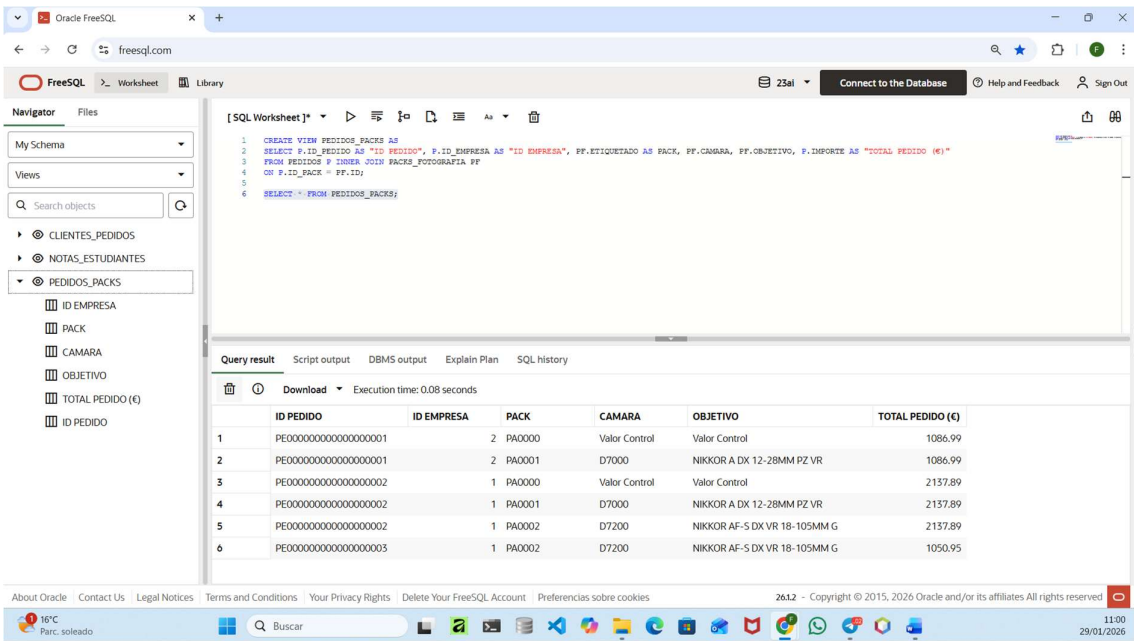
View CLIENTES_PEDIDOS created.

Elapsed: 00:00:00.012

SQL> CREATE VIEW PEDIDOS_PACKS AS
SELECT P.ID_PEDIDO AS "ID PEDIDO", P.ID_EMPRESA AS "ID EMPRESA", PF.ETIQUETADO AS PACK, PF.CAMARA, PF.OBJETIVO, P.IMPORTE AS "TOTAL PEDIDO (€)"
FROM PEDIDOS P INNER JOIN PACKS_FOTOGRAFIA PF
ON P.ID_PACK = PF.ID;

View PEDIDOS_PACKS created.

Elapsed: 00:00:00.011



Oracle FreeSQL

Navigator

My Schema

Views

Search objects

CLIENTES_PEDIDOS

NOTAS_ESTUDIANTES

PEDIDOS_PACKS

ID EMPRESA

PACK

CAMARA

OBJETIVO

TOTAL PEDIDO (€)

ID PEDIDO

[SQL Worksheet]

```
1 CREATE VIEW PEDIDOS_PACKS AS
2 SELECT P.ID_PEDIDO AS "ID PEDIDO", P.ID_EMPRESA AS "ID EMPRESA", PF.ETIQUETADO AS PACK, PF.CAMARA, PF.OBJETIVO, P.IMPORTE AS "TOTAL PEDIDO (€)"
3 FROM PEDIDOS P INNER JOIN PACKS_FOTOGRAFIA PF
4 ON P.ID_PACK = PF.ID;
5
6 SELECT * FROM PEDIDOS_PACKS;
```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.08 seconds

	ID PEDIDO	ID EMPRESA	PACK	CAMARA	OBJETIVO	TOTAL PEDIDO (€)
1	PE000000000000000000000001	2	PA0000	Valor Control	Valor Control	1086.99
2	PE000000000000000000000001	2	PA0001	D7000	NIKOR A DX 12-28MM PZ VR	1086.99
3	PE000000000000000000000002	1	PA0000	Valor Control	Valor Control	2137.89
4	PE000000000000000000000002	1	PA0001	D7000	NIKOR A DX 12-28MM PZ VR	2137.89
5	PE000000000000000000000002	1	PA0002	D7200	NIKOR AF-S DX VR 18-105MM G	2137.89
6	PE000000000000000000000003	1	PA0002	D7200	NIKOR AF-S DX VR 18-105MM G	1050.95

The screenshot shows the Oracle FreeSQL web interface. The top navigation bar includes the 'FreeSQL' logo and a 'Worksheet' tab. The left sidebar contains a 'Navigator' panel with 'My Schema' and 'Views' sections. The main area displays a SQL script in a 'SQL Worksheet' editor. The script creates a view named 'TODAS_TABLAS' by joining several tables: PEDIDOS, CLIENTES, PACES_FOTOGRAFIA, STOCKS, and EMPRESAS. The script includes comments and uses aliases for clarity. Below the editor, the 'Query result' tab is active, showing the execution status: 'View TODAS_TABLAS created.' and 'Elapsed: 00:00:00.012'.

6

3.- Crear los siguientes procedimientos: (1 PUNTOS CADA UNA)

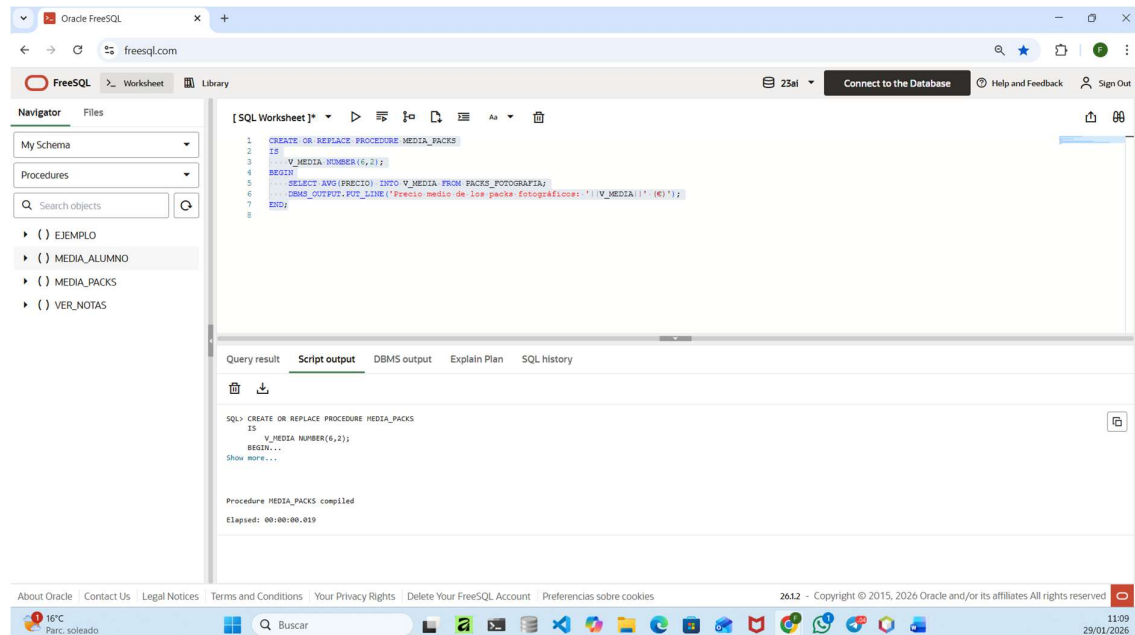
ESTE PROCEDIMIENTO MOSTRARÁ POR CONSOLA UNA SUMA O MEDIA DE UNO DE LOS CAMPOS NUMÉRICOS.

- PEGA AQUÍ LAS SENTENCIAS SQL -

> Procedimiento que calcula el **precio medio de los tipos de packs disponibles**:

```
CREATE OR REPLACE PROCEDURE MEDIA_PACKS
IS
  V_MEDIA NUMBER(6,2);
BEGIN
  SELECT AVG(PRECIO) INTO V_MEDIA FROM PACKS_FOTOGRAFIA;
  DBMS_OUTPUT.PUT_LINE('Precio medio de los packs fotográficos: ' || V_MEDIA || ' (€)');
END;
```

- PEGA AQUÍ LAS CAPTURA DE PANTALLA COMPLETAS QUE MUESTREN SU CORRECTA EJECUCIÓN -



The screenshot displays the Oracle FreeSQL web application interface. The browser address bar shows 'freesql.com'. The interface includes a 'Navigator' panel on the left with a tree view containing 'My Schema', 'Procedures', and a search bar. The main workspace is titled '[SQL Worksheet]*' and contains a SQL script:

```
1 CREATE OR REPLACE PROCEDURE MEDIA_PACKS
2 IS
3   V_MEDIA NUMBER(6,2);
4 BEGIN
5   SELECT AVG(PRECIO) INTO V_MEDIA FROM PACKS_FOTOGRAFIA;
6   DBMS_OUTPUT.PUT_LINE('Precio medio de los packs fotograficos: ' || V_MEDIA || ' (€)');
7 END;
8
9 EXEC MEDIA_PACKS;
```

Below the script, the 'Script output' tab is active, showing the execution results:

```
Procedure MEDIA_PACKS compiled
Elapsed: 00:00:00.019

SQL> EXEC MEDIA_PACKS

Precio medio de los packs fotograficos: 534.47 (€)

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.009
```

The bottom of the interface features a footer with links like 'About Oracle', 'Contact Us', and 'Legal Notices', along with a copyright notice for 2015-2026 Oracle. The Windows taskbar at the very bottom shows the date as 29/01/2026 and the time as 11:10.

ESTE PROCEDIMIENTO MOSTRARÁ TODOS LOS REGISTROS DE UNA TABLA QUE CUMPLAN CON UNA CONDICIÓN QUE SE ESTABLECERÁ POR PARÁMETRO. USA UN CURSOR EXPLÍCITO.

- PEGA AQUÍ LAS SENTENCIAS SQL -

> Procedimiento que muestra **todos los registros de la tabla PACKS_FOTOGRAFIA** cuyo **precio es superior a una cantidad** introducida como **parámetro**:

```
CREATE OR REPLACE PROCEDURE PACKS_PRECIO_SUPERIOR(P_PRECIO NUMBER)
IS
  CURSOR cPacks
  IS
    SELECT ID, ETIQUETADO, MARCA, CAMARA, OBJETIVO, PRECIO, FECHA_CREACION FROM PACKS_FOTOGRAFIA WHERE
    PRECIO>P_PRECIO;

  V_ID NUMBER(4);
  V_ETIQUETADO CHAR(6);
  V_MARCA VARCHAR2 (20);
  V_CAMARA VARCHAR2 (20);
  V_OBJETIVO VARCHAR2 (50);
  V_PRECIO NUMBER(6,2);
  V_FECHA_CREACION DATE;
BEGIN
  OPEN cPacks;

  LOOP

    FETCH cPacks INTO V_ID, V_ETIQUETADO, V_MARCA, V_CAMARA, V_OBJETIVO, V_PRECIO, V_FECHA_CREACION;
    EXIT WHEN cPacks%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('ID PACK: '||V_ID||', ETIQUETA: '||V_ETIQUETADO||', MARCA: '||V_MARCA||', CÁMARA:
    '||V_CAMARA||', OBJETIVO: '||V_OBJETIVO||', PRECIO: '||V_PRECIO||' (€), FECHA DEL PACK: '||V_FECHA_CREACION);

    END LOOP;

  CLOSE cPacks;
EXCEPTION
  WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('ERROR');
END;
```

- PEGA AQUÍ LAS CAPTURA DE PANTALLA COMPLETAS QUE MUESTREN SU CORRECTA EJECUCIÓN -

The first screenshot shows the creation of a PL/SQL procedure named `PACKS_PRECIO_SUPERIOR`. The code defines a cursor `cPacks` that selects product details from the `PACKS_FOTOGRAFIA` table where the price is greater than a given parameter. The procedure iterates through these products, displaying their details using `DBMS_OUTPUT.PUT_LINE`.

```

1 CREATE OR REPLACE PROCEDURE PACKS_PRECIO_SUPERIOR(P_PRECIO NUMBER)
2 IS
3
4     CURSOR cPacks
5     IS
6         SELECT ID, ETIQUETADO, MARCA, CÁMARA, OBJETIVO, PRECIO, FECHA_CREACION FROM PACKS_FOTOGRAFIA WHERE PRECIO > P_PRECIO;
7
8     V_ID NUMBER(4);
9     V_ETIQUETADO CHAR(6);
10    V_MARCA VARCHAR2(20);
11    V_CÁMARA VARCHAR2(20);
12    V_OBJETIVO VARCHAR2(50);
13    V_PRECIO NUMBER(4,2);
14    V_FECHA_CREACION DATE;
15
16 BEGIN
17     OPEN cPacks;
18     LOOP
19         FETCH cPacks INTO V_ID, V_ETIQUETADO, V_MARCA, V_CÁMARA, V_OBJETIVO, V_PRECIO, V_FECHA_CREACION;
20         EXIT WHEN cPacks%NOTFOUND;
21         DBMS_OUTPUT.PUT_LINE('ID PACK: '||V_ID||', ETIQUETA: '||V_ETIQUETADO||', MARCA: '||V_MARCA||', CÁMARA: '||V_CÁMARA||', OBJETIVO: '||V_OBJETIVO||', PRECIO: '||V_
22         PRECIO||', FECHA CREACION: '||V_FECHA_CREACION||');
23     END LOOP;
24     CLOSE cPacks;
25 EXCEPTION
26     WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('ERROR');
27 END;

```

The second screenshot shows the execution of the procedure with a price of 600. The output displays the details of two products: a Nikon D7000 camera and a Canon EOS 250D camera.

```

29
30
31 EXEC PACKS_PRECIO_SUPERIOR(600);

```

Procedure PACKS_PRECIO_SUPERIOR compiled
Elapsed: 00:00:00.011

SQL> EXEC PACKS_PRECIO_SUPERIOR(600)

ID PACK: 2, ETIQUETA: PANN01, MARCA: NIKON, CÁMARA: D7000, OBJETIVO: NIKKOR A DX 12-200mm F2 VR, PRECIO: 655.95 (€), FECHA DEL PACK: 13-MAY-21
ID PACK: 4, ETIQUETA: PANN03, MARCA: CANON, CÁMARA: EOS 250D, OBJETIVO: EF 180-400 mm · f/4.5-5.6 L 25 USM, PRECIO: 1050.95 (€), FECHA DEL PACK: 10-JAN-20

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.008

Oracle FreeSQL

freesql.com

FreeSQL

Worksheet

Library

23s

Connect to the Database

Help and Feedback

Sign Out

Navigator

Files

My Schema

Procedures

Search objects

() EJEMPLO

() MEDIA_ALUMNO

() MEDIA_PACKS

() PACKS_PRECIO_SUPERIOR

() VER_NOTAS

[SQL Worksheet]*

29

30

31 EXEC PACKS_PRECIO_SUPERIOR(600);

32

33 EXEC PACKS_PRECIO_SUPERIOR(100);

Query result

Script output

DBMS output

Explain Plan

SQL history

ID PACK: 4, ETIQUETA: PANN03, MARCA: CANON, CÁMARA: EOS 250D, OBJETIVO: EF 100-400 mm - f/4.5-5.6 L IS USM, PRECIO: 1850.95 (€), FECHA DEL PACK: 18-3AB-20

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.008

SQL> EXEC PACKS_PRECIO_SUPERIOR(100)

ID PACK: 2, ETIQUETA: PANN01, MARCA: NIKON, CÁMARA: D7000, OBJETIVO: NIKKOR A DX 12-28MM F2 VR, PRECIO: 655.95 (€), FECHA DEL PACK: 13-MAY-21

ID PACK: 3, ETIQUETA: PANN02, MARCA: NIKON, CÁMARA: D7200, OBJETIVO: NIKKOR AF-S DX VR 18-105MM G, PRECIO: 430.99 (€), FECHA DEL PACK: 11-MAR-22

ID PACK: 4, ETIQUETA: PANN03, MARCA: CANON, CÁMARA: EOS 250D, OBJETIVO: EF 100-400 mm - f/4.5-5.6 L IS USM, PRECIO: 1850.95 (€), FECHA DEL PACK: 18-3AB-20

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.004

About Oracle

Contact Us

Legal Notices

Terms and Conditions

Your Privacy Rights

Delete Your FreeSQL Account

Preferencias sobre cookies

2012 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

17°C

Parc: soleado

Buscar

11:42

29/01/2026

4.- Crear un JOB que ejecute un PROCEDIMIENTO que inserte un registro en una tabla cada 10 minutos. (1 PUNTO)

- PEGA AQUÍ LAS SENTENCIAS SQL -

> En primer lugar creamos un **procedimiento** que realiza una **inserción de un registro tipo** en la tabla pedidos con parámetros definidos que podrían usarse a posteriori para llevar a cabo acciones de control sobre esa tabla. Cabe indicar que para facilitar estas tareas se han insertado en las tablas **PACKS_FOTOGRAFIA** y **CLIENTES** un primer registro con datos ficticios de control (ver apartado de inserts del punto 1).

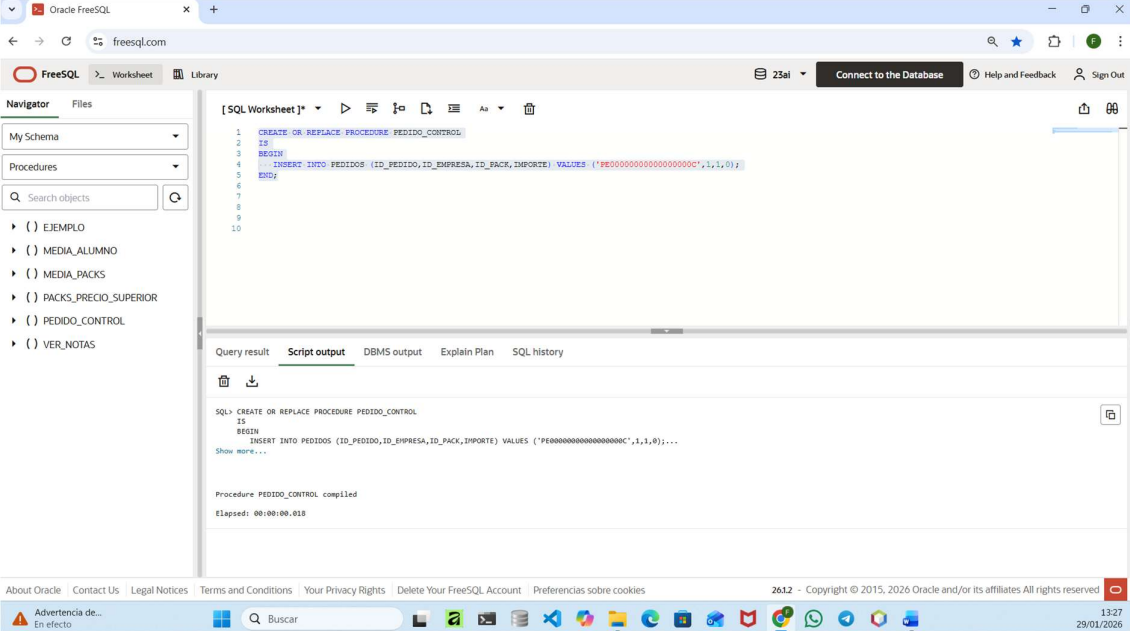
```
CREATE OR REPLACE PROCEDURE PEDIDO_CONTROL
IS
BEGIN
    INSERT INTO PEDIDOS (ID_PEDIDO,ID_EMPRESA,ID_PACK,IMPORTE) VALUES ('PE000000000000000000C',1,1,0);
END;
```

> Seguidamente creamos el **JOB 'PEDIDO_CONTROL'** que ejecutará el procedimiento anterior cada 10 minutos, estando activo desde su momento de creación:

```
BEGIN
    DBMS_SCHEDULER.create_job (
        job_name      => 'INSERTO_PEDIDO_CONTROL',
        job_type       => 'STORED_PROCEDURE',
        job_action      => 'PEDIDO_CONTROL',
        start_date      => SYSTIMESTAMP,
        repeat_interval => 'FREQ=MINUTELY; INTERVAL=10',
        enabled         => TRUE,
        comments        => 'Inserto de pedido control en registro - cada 10min'
    );
END;
```

- PEGA AQUÍ LAS CAPTURA DE PANTALLA COMPLETAS QUE MUESTREN SU CORRECTA EJECUCIÓN -

> Creación del procedimiento:



The screenshot shows the Oracle FreeSQL web interface. In the SQL Worksheet, the following SQL code is entered:

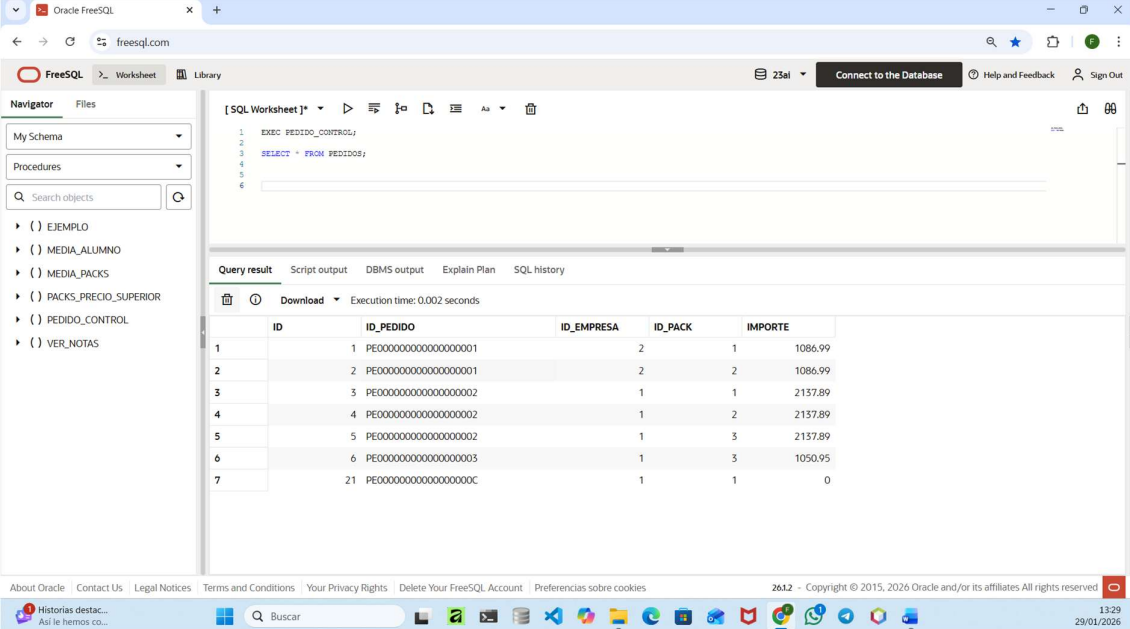
```
1 CREATE OR REPLACE PROCEDURE PEDIDO_CONTROL
2 IS
3 BEGIN
4   INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000000000000000000', 1, 1, 0);
5 END;
```

The Script output tab shows the successful execution of the procedure:

```
SQL> CREATE OR REPLACE PROCEDURE PEDIDO_CONTROL
15
BEGIN
INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000000000000000000', 1, 1, 0); ...
Show more...

Procedure PEDIDO_CONTROL compiled
Elapsed: 00:00:00.018
```

> Comprobación de que el procedimiento funciona:



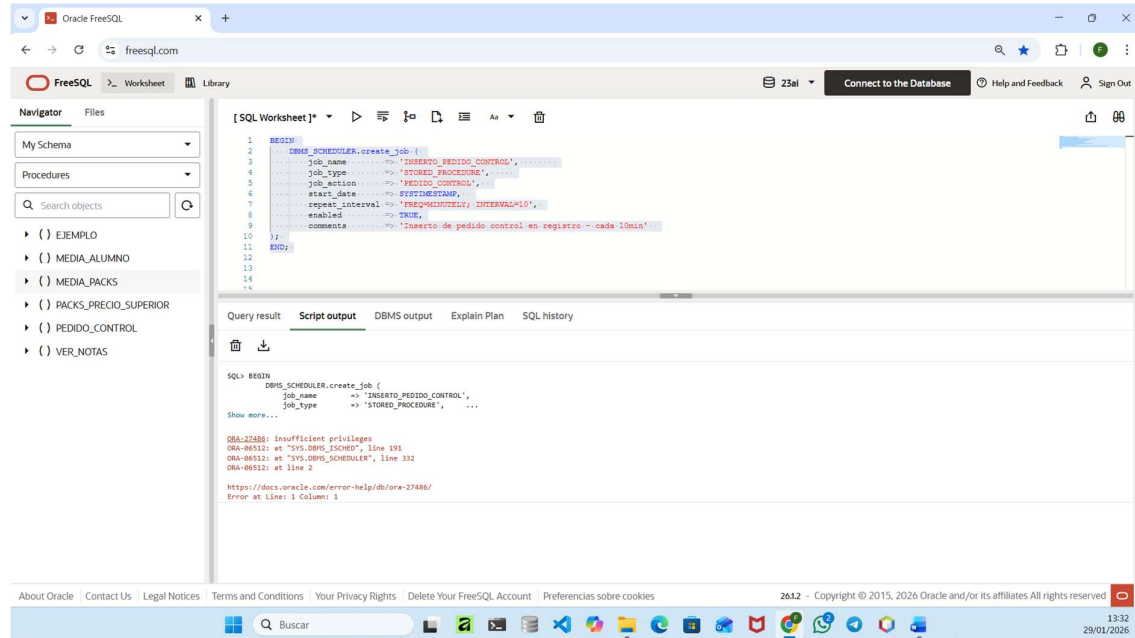
The screenshot shows the Oracle FreeSQL web interface with the following SQL code in the worksheet:

```
1 EXEC PEDIDO_CONTROL;
2
3 SELECT * FROM PEDIDOS;
4
5
6
```

The Query result tab displays the output of the SELECT statement, showing a table with 7 rows and 5 columns: ID, ID_PEDIDO, ID_EMPRESA, ID_PACK, and IMPORTE.

	ID	ID_PEDIDO	ID_EMPRESA	ID_PACK	IMPORTE
1	1	PE00000000000000000000000000000000	2	1	1086.99
2	2	PE00000000000000000000000000000000	2	2	1086.99
3	3	PE00000000000000000000000000000000	1	1	2137.89
4	4	PE00000000000000000000000000000000	1	2	2137.89
5	5	PE00000000000000000000000000000000	1	3	2137.89
6	6	PE00000000000000000000000000000000	1	3	1050.95
7	21	PE00000000000000000000000000000000	1	1	0

> **JOB:**



> Observamos que no es posible crearlo en la versión Live de Oracle al no tener privilegios para ello.

5.- Crear mediante trigger un sistema que guarde un control de uso de una tabla. El sistema deberá reflejar que se ha insertado un registro, modificado o eliminado con sus datos y la fecha en la que se realiza la operación. No se admite multitrigger. Este enunciado se explicará en clase con un ejemplo. (2 PUNTOS)

- PEGA AQUÍ LAS SENTENCIAS SQL -

> Creamos una tabla en la que se pretende llevar un control sencillo sobre la tabla **PEDIDOS**; la idea es que cada vez que se añada, actualice o borre un registro de la tabla PEDIDOS se refleje en una tabla que llamaremos **CONTROL_PEDIDOS**, en la que simplemente almacenaremos el ID del pedido sobre el que se ha actuado y la operación realizada:

```
CREATE TABLE CONTROL_PEDIDOS (  
    ID NUMBER(6) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    ID_PEDIDO CHAR (20) NOT NULL,  
    OPERACION VARCHAR2 (10) NOT NULL,  
    FECHA DATE  
);
```

> **Primer trigger:** actualiza la tabla CONTROL_PEDIDOS cuando se **inserte** un registro en PEDIDOS.

```
CREATE OR REPLACE TRIGGER CTRL_INSERT_PEDIDOS  
AFTER INSERT ON PEDIDOS  
FOR EACH ROW  
BEGIN  
    INSERT INTO CONTROL_PEDIDOS (ID_PEDIDO,OPERACION,FECHA) VALUES (:NEW.ID_PEDIDO,'INSERTADO',SYSDATE);  
END;
```

> **Segundo trigger:** actualiza la tabla CONTROL_PEDIDOS cuando se **actualiza** algún registro en PEDIDOS.

```
CREATE OR REPLACE TRIGGER CTRL_UPDATE_PEDIDOS  
AFTER UPDATE ON PEDIDOS  
FOR EACH ROW  
BEGIN  
    INSERT INTO CONTROL_PEDIDOS (ID_PEDIDO,OPERACION,FECHA) VALUES (:OLD.ID_PEDIDO,'MODIFICADO',  
SYSDATE);  
END;
```

> **Tercer trigger:** actualiza la tabla CONTROL_PEDIDOS cuando se **borra** algún registro en PEDIDOS.

```
CREATE OR REPLACE TRIGGER CTRL_DELETE_PEDIDOS  
BEFORE DELETE ON PEDIDOS  
FOR EACH ROW  
BEGIN  
    INSERT INTO CONTROL_PEDIDOS (ID_PEDIDO,OPERACION,FECHA) VALUES (:OLD.ID_PEDIDO,'BORRADO', SYSDATE);  
END;
```

- PEGA AQUÍ LAS CAPTURA DE PANTALLA COMPLETAS QUE MUESTREN SU CORRECTA EJECUCIÓN -

> Creación de la **tabla para control**:

The screenshot shows the Oracle FreeSQL web interface. In the left sidebar, the 'Navigator' pane shows a tree view with 'My Schema' expanded, listing tables: CLIENTES, CONTROL_PEDIDOS, ESTUDIANTES, MATERIAS, NOTAS, PACKS_FOTOGRAFIA, PEDIDOS, and STOCKS. The main editor area, titled '[SQL Worksheet]*', contains the following SQL code:

```
1 CREATE TABLE CONTROL_PEDIDOS (  
2   ID_PEDIDO(6) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3   ID_PEDIDO CHAR(20) NOT NULL,  
4   OPERACION VARCHAR2(10) NOT NULL,  
5   FECHA DATE  
6 )  
7  
8  
9  
10  
11
```

Below the editor, the 'Script output' pane shows the execution results:

```
ID_PEDIDO CHAR(20) NOT NULL,  
OPERACION VARCHAR2(10) NOT NULL,...  
Show more...  
  
Table CONTROL_PEDIDOS created.  
Elapsed: 00:00:00.027
```

The bottom status bar indicates '26.1.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved' and the date '29/01/2026'.

> **Primer trigger**: actualiza la tabla CONTROL_PEDIDOS cuando se inserte un registro en PEDIDOS.

The screenshot shows the Oracle FreeSQL web interface. In the left sidebar, the 'Navigator' pane shows a tree view with 'My Schema' expanded, listing triggers: CTRL_INSERT_PEDIDOS. The main editor area, titled '[SQL Worksheet]*', contains the following SQL code:

```
1 CREATE OR REPLACE TRIGGER CTRL_INSERT_PEDIDOS  
2 AFTER INSERT ON PEDIDOS  
3 FOR EACH ROW  
4 BEGIN  
5   INSERT INTO CONTROL_PEDIDOS (ID_PEDIDO, OPERACION, FECHA) VALUES (NEW.ID_PEDIDO, 'INSERTADO', SYSDATE);  
6 END;
```

Below the editor, the 'Script output' pane shows the execution results:

```
SQL> CREATE OR REPLACE TRIGGER CTRL_INSERT_PEDIDOS  
AFTER INSERT ON PEDIDOS  
FOR EACH ROW  
BEGIN...  
Show more...  
  
Trigger CTRL_INSERT_PEDIDOS compiled  
Elapsed: 00:00:00.013
```

The bottom status bar indicates '26.1.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved' and the date '29/01/2026'.

The screenshot displays the Oracle FreeSQL web application interface. The browser address bar shows 'freesql.com'. The interface includes a 'Navigator' on the left with 'My Schema' and 'Triggers' sections, and a search bar. The main area is titled '[SQL Worksheet]*' and contains the following SQL code:

```
1 INSERT INTO PEDIDOS (ID_PEDIDO, ID_EMPRESA, ID_PACK, IMPORTE) VALUES ('PE00000000000000000004', 3, 2, 430.99);  
2 SELECT * FROM PEDIDOS;  
3 SELECT * FROM CONTROL_PEDIDOS;  
4  
5
```

Below the code editor, the 'Query result' tab is active, showing the execution time as 0.007 seconds. The results are displayed in a table with the following columns: ID, ID_PEDIDO, OPERACION, and FECHA.

ID	ID_PEDIDO	OPERACION	FECHA
1	PE00000000000000000004	INSERTADO	1/29/2026, 12:54:24 PM

The bottom of the interface shows a footer with links for 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', 'Delete Your FreeSQL Account', and 'Preferencias sobre cookies'. The version number '261.2' and copyright information 'Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved' are also present. The Windows taskbar at the bottom shows the date '29/01/2026' and time '13:55'.

> **Segundo trigger:** actualiza la tabla CONTROL_PEDIDOS cuando se actualiza algún registro en PEDIDOS.

The screenshot shows the Oracle FreeSQL web interface. In the left sidebar, the 'Triggers' section is expanded, showing two triggers: CTRL_INSERT_PEDIDOS and CTRL_UPDATE_PEDIDOS. The main editor area contains the following SQL code:

```
1 CREATE OR REPLACE TRIGGER CTRL_UPDATE_PEDIDOS
2 AFTER UPDATE ON PEDIDOS
3 FOR EACH ROW
4 BEGIN
5     INSERT INTO CONTROL_PEDIDOS (ID_PEDIDO, OPERACION, FECHA) VALUES ((OLD.ID_PEDIDO, 'MODIFICADO', SYSDATE));
6 END;
```

Below the editor, the 'Script output' tab is active, displaying the execution results:

```
SQL> CREATE OR REPLACE TRIGGER CTRL_UPDATE_PEDIDOS
AFTER UPDATE ON PEDIDOS
FOR EACH ROW
BEGIN...
Show more...

Trigger CTRL_UPDATE_PEDIDOS compiled
Elapsed: 00:00:00.021
```

> Comprobación de funcionamiento modificando un registro y visualizando las dos tablas afectadas:

The screenshot shows the Oracle FreeSQL web interface. The main editor area contains the following SQL code:

```
1 UPDATE PEDIDOS SET IMPORTE=431 WHERE IMPORTE=430.99;
2 SELECT * FROM PEDIDOS;
3 SELECT * FROM CONTROL_PEDIDOS;
```

Below the editor, the 'Script output' tab is active, displaying the execution results:

```
SQL> UPDATE PEDIDOS SET IMPORTE=431 WHERE IMPORTE=430.99

1 row updated.
Elapsed: 00:00:00.011
```

Oracle FreeSQL

freessql.com

FreeSQL

Worksheet

Library

23s

Connect to the Database

Help and Feedback

Sign Out

Navigator

Files

My Schema

Triggers

Search objects

CTRL_INSERT_PEDIDOS

CTRL_UPDATE_PEDIDOS

[SQL Worksheet]*

1 UPDATE PEDIDOS SET IMPORTE=431 WHERE IMPORTE=430.99;

2 SELECT * FROM PEDIDOS;

3 SELECT * FROM CONTROL_PEDIDOS;

4

5

6

7

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.005 seconds

ID	ID_PEDIDO	ID_EMPRESA	ID_PACK	IMPORTE
1	1	PE00000000000000000001	2	1086.99
2	2	PE00000000000000000001	2	1086.99
3	3	PE00000000000000000002	1	2137.89
4	4	PE00000000000000000002	1	2137.89
5	5	PE00000000000000000002	1	2137.89
6	6	PE00000000000000000003	1	1050.95
7	42	PE00000000000000000004	3	431

About Oracle

Contact Us

Legal Notices

Terms and Conditions

Your Privacy Rights

Delete Your FreeSQL Account

Preferencias sobre cookies

26.12 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

20°C

Mayorm. nublado

Buscar

13:58

29/01/2026

Oracle FreeSQL

freessql.com

FreeSQL

Worksheet

Library

23s

Connect to the Database

Help and Feedback

Sign Out

Navigator

Files

My Schema

Triggers

Search objects

CTRL_INSERT_PEDIDOS

CTRL_UPDATE_PEDIDOS

[SQL Worksheet]*

1 UPDATE PEDIDOS SET IMPORTE=431 WHERE IMPORTE=430.99;

2 SELECT * FROM PEDIDOS;

3 SELECT * FROM CONTROL_PEDIDOS;

4

5

6

7

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.002 seconds

ID	ID_PEDIDO	OPERACION	FECHA
1	1	INSERTADO	1/29/2026, 12:54:24 PM
2	21	MODIFICADO	1/29/2026, 12:58:06 PM

About Oracle

Contact Us

Legal Notices

Terms and Conditions

Your Privacy Rights

Delete Your FreeSQL Account

Preferencias sobre cookies

26.12 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

20°C

Mayorm. nublado

Buscar

13:59

29/01/2026

> **Tercer trigger:** actualiza la tabla CONTROL_PEDIDOS cuando se borra algún registro en PEDIDOS.

The screenshot shows the Oracle FreeSQL web interface. In the left sidebar, the 'Triggers' section is expanded, showing three triggers: CTRL_DELETE_PEDIDOS, CTRL_INSERT_PEDIDOS, and CTRL_UPDATE_PEDIDOS. The main editor area displays the SQL code for creating the CTRL_DELETE_PEDIDOS trigger. The script output pane shows the execution results, indicating that 1 row was updated and the trigger was compiled successfully.

```

1 CREATE OR REPLACE TRIGGER CTRL_DELETE_PEDIDOS
2 BEFORE DELETE ON PEDIDOS
3 FOR EACH ROW
4 BEGIN
5     INSERT INTO CONTROL_PEDIDOS (ID_PEDIDO, OPERACION, FECHA) VALUES (1000.ID_PEDIDO, 'BORRADO', SYSDATE);
6 END;
7
8
9
10
11

```

Query result: Script output DBMS output Explain Plan SQL history

1 row updated.
Elapsed: 00:00:00.011

SQL> CREATE OR REPLACE TRIGGER CTRL_DELETE_PEDIDOS
BEFORE DELETE ON PEDIDOS
FOR EACH ROW
BEGIN...

Show more...

Trigger CTRL_DELETE_PEDIDOS compiled
Elapsed: 00:00:00.014

> Comprobación de funcionamiento borrando un registro y visualizando las dos tablas afectadas:

The screenshot shows the Oracle FreeSQL web interface. The main editor area displays the SQL code for deleting a record from the PEDIDOS table. The script output pane shows the execution results, indicating that 1 row was deleted. The left sidebar shows the same three triggers as in the previous screenshot.

```

1 DELETE PEDIDOS WHERE ID_PEDIDO = 'PE00000000000000000004';
2
3 SELECT * FROM PEDIDOS;
4 SELECT * FROM CONTROL_PEDIDOS;
5
6
7
8
9

```

Query result: Script output DBMS output Explain Plan SQL history

SQL> DELETE PEDIDOS WHERE ID_PEDIDO = 'PE00000000000000000004'

1 row deleted.
Elapsed: 00:00:00.009

The screenshot shows the Oracle FreeSQL web interface. The SQL Worksheet contains the following query:

```
1 DELETE PEDIDOS WHERE ID_PEDIDO = 'PE00000000000000000004';
2 SELECT * FROM PEDIDOS;
3 SELECT * FROM CONTROL_PEDIDOS;
4
5
6
7
8
9
```

The Query result tab shows the following table:

ID	ID_PEDIDO	ID_EMPRESA	ID_PACK	IMPORTE
1	1 PE00000000000000000001	2	1	1086.99
2	2 PE00000000000000000001	2	2	1086.99
3	3 PE00000000000000000002	1	1	2137.89
4	4 PE00000000000000000002	1	2	2137.89
5	5 PE00000000000000000002	1	3	2137.89
6	6 PE00000000000000000003	1	3	1050.95

The screenshot shows the Oracle FreeSQL web interface. The SQL Worksheet contains the following query:

```
1 DELETE PEDIDOS WHERE ID_PEDIDO = 'PE00000000000000000004';
2 SELECT * FROM PEDIDOS;
3 SELECT * FROM CONTROL_PEDIDOS;
4
5
6
7
8
9
```

The Query result tab shows the following table:

ID	ID_PEDIDO	OPERACION	FECHA
1	1 PE00000000000000000004	INSERTADO	1/29/2026, 12:54:21
2	21 PE00000000000000000004	MODIFICADO	1/29/2026, 12:58:01
3	22 PE00000000000000000004	BORRADO	1/29/2026, 1:01:43

IMPORTANTE:

Se entrega un único documento en pdf, no se revisarán trabajos en otro formato.

Si la captura de pantalla no se adjunta o no es de pantalla completa con fecha y hora a la vista se anulará el 100% del valor de la pregunta.

No se admite uso de IA ni sintaxis diferente a la vista en clase, en caso contrario se calificará el ejercicio con un cero.